UNIVERSITÉ DE YAOUNDÉ 1

Faculté des Sciences

Département d'Informatique

Master 2 – INF5039 : Ingénierie Dirigée par les Modèles

PROJET

Modélisation et génération d'un mécanisme de répartition de charge par DSL pour un système distribué

Réalisé par :

ZOGO ABOUMA Zozime Achaire – 18N2824

NKOLO Stéphane Roylex – 21T2588

Encadreur :

Dr MONTHE Valérie

Université de Yaoundé 1

Année académique 2025 – 2026

Table des matières

1 Introduction **3**

2 Description du domaine traité **4**
2.1 Qu'est-ce que la répartition de charge ? 4
2.2 Composants d'un système de répartition de charge 4
2.3 Algorithmes de répartition courants 5
2.4 Contexte et problématique 5

3 Tableau des concepts identifiés **6**

4 Règles de gestion et exigences **7**

5 Décomposition en modules **8**
5.1 Architecture des packages 8
5.2 Description des modules 9

6 Schéma du métamodèle **10**
6.1 Diagramme global du métamodèle 10
6.2 Module Core – Détail 10

7 Contraintes OCL **13**
7.1 RG01 – Unicité des noms de serveurs 13
7.2 RG02 – Port valide 13
7.3 RG03 – Poids positif 13
7.4 RG04 – Cluster non vide 13
7.5 RG05 – Health check cohérent 14
7.6 RG06 – Seuils de scaling cohérents 14
7.7 RG07 – Instances min/max cohérentes 14
7.8 RG08 – Listener unique par port 14
7.9 RG09 – Algorithme Weighted requiert des poids 14
7.10 RG10 – Session Persistence cookie 14
7.11 RG11 – Au moins un listener 15
7.12 RG12 – Serveur actif dans le cluster 15

8 Niveaux MDA et architecture de l'approche **16**
8.1 Les niveaux MDA pris en compte 16
8.2 Schéma d'architecture de l'approche 17
8.3 Justification des choix technologiques 17

9 Spécification des transformations **18**

9.1 Vue d'ensemble . . . 18
9.2 Définition de la grammaire Xtext (syntaxe concrète) . . . 18
9.2.1 Explication de la grammaire . . . 21
9.2.2 Artefacts générés par Xtext . . . 22
9.3 Transformation T1 : PIM → PSM Nginx (ATL) . . . 22
9.4 Transformation T3 : PSM Nginx → Code (Acceleo) . . . 23
9.5 Transformation T5 : PIM → Docker Compose (Acceleo) . . . 24

10 Installation et configuration des outils 25
10.1 Prérequis système . . . 25
10.2 Installation de Docker . . . 25
10.2.1 Sur Ubuntu/Debian . . . 25
10.2.2 Sur macOS . . . 26
10.3 Installation de Docker Compose . . . 26
10.4 Installation de Nginx (pour tests locaux sans Docker) . . . 27
10.5 Installation des outils IDM (Eclipse) . . . 27
10.6 Outils complémentaires utiles . . . 27
10.7 Vérification de l'installation complète . . . 28

11 Étude de cas : Application E-Commerce 29
11.1 Description du cas d'étude . . . 29
11.2 Modèle DSL du cas d'étude . . . 29
11.3 Code généré : nginx.conf . . . 31
11.4 Code généré : docker-compose.yml . . . 32
11.5 Déploiement et test . . . 34
11.5.1 Lancement de l'infrastructure . . . 34
11.5.2 Tests de la répartition de charge . . . 34
11.5.3 Vérification du health check . . . 35

12 Tutoriel de réalisation pas à pas 36
12.1 Prérequis . . . 36
12.2 Étape 1 : Création du métamodèle Ecore . . . 36
12.3 Étape 2 : Génération du code Java EMF . . . 36
12.4 Étape 3 : Définition de la syntaxe concrète (Xtext) . . . 36
12.5 Étape 4 : Implémentation des contraintes . . . 37
12.6 Étape 5 : Transformations ATL . . . 37
12.7 Étape 6 : Génération de code (Acceleo) . . . 37
12.8 Étape 7 : Test de bout en bout . . . 37

13 Perspectives et améliorations 38

Conclusion 39

Références bibliographiques 40

Chapitre 1

Introduction

Dans le contexte actuel des systèmes informatiques, la montée en charge des applications distribuées impose des mécanismes efficaces de répartition de charge (*load balancing*). Ces mécanismes permettent de distribuer équitablement les requêtes entre plusieurs serveurs afin d'optimiser les performances, d'assurer la haute disponibilité et de prévenir la surcharge de nœuds individuels.

L'Ingénierie Dirigée par les Modèles (IDM/MDE) offre une approche systématique pour modéliser, transformer et générer des artefacts logiciels à partir de modèles abstraits. En combinant IDM et la définition d'un langage dédié (DSL – *Domain Specific Language*), il devient possible de spécifier de manière déclarative les politiques de répartition de charge et de générer automatiquement le code d'infrastructure correspondant.

Ce projet a pour objectif de concevoir un DSL permettant de modéliser un mécanisme de répartition de charge pour un système distribué, puis de mettre en œuvre une chaîne de transformations MDA (*Model Driven Architecture*) allant du modèle abstrait jusqu'au code exécutable.

Chapitre 2

Description du domaine traité

2.1 Qu'est-ce que la répartition de charge ?

La répartition de charge (*load balancing*) est une technique utilisée dans les systèmes distribués pour distribuer le trafic réseau ou les tâches de calcul entre plusieurs serveurs ou ressources. L'objectif est de :

- Maximiser le débit (*throughput*) du système global
- Minimiser le temps de réponse pour chaque requête
- Éviter la surcharge d'un seul nœud
- Assurer la tolérance aux pannes (*failover*)
- Permettre la scalabilité horizontale

2.2 Composants d'un système de répartition de charge

Un système de répartition de charge typique comprend les éléments suivants :

- **Load Balancer** (répartiteur) : point d'entrée qui reçoit les requêtes et les redirige
- **Backend Servers** (serveurs backend) : ensemble de serveurs qui traitent les requêtes
- **Health Check** (vérification de santé) : mécanisme pour vérifier la disponibilité des serveurs
- **Algorithme de répartition** : stratégie utilisée pour choisir le serveur cible
- **Session Persistence** (affinité de session) : mécanisme pour diriger un même client vers le même serveur
- **Cluster** : regroupement logique de serveurs backend
- **Listeners** : points d'écoute sur des ports/protocoles spécifiques

2.3 Algorithmes de répartition courants

Table 2.1 – Algorithmes de répartition de charge

Algorithme	**Description**	**Cas d'usage**
Round Robin	Distribution séquentielle cyclique entre les serveurs	Serveurs homogènes
Weighted Round Robin	Round Robin avec poids assignés	Serveurs hétérogènes
Least Connections	Envoie au serveur ayant le moins de connexions actives	Requêtes de durée variable
IP Hash	Hash de l'IP source pour déterminer le serveur	Affinité de session
Random	Choix aléatoire	Cas simples, tests
Weighted Least Conn.	Least Connections pondéré par la capacité	Serveurs hétérogènes, charge variable

2.4 Contexte et problématique

Traditionnellement, la configuration d'un load balancer se fait manuellement via des fichiers de configuration (nginx.conf, haproxy.cfg, etc.) qui sont :

— Spécifiques à une technologie (*vendor lock-in*)

— Sujets à des erreurs de syntaxe

— Difficiles à valider avant déploiement

— Non portables d'une plateforme à l'autre

Notre approche IDM propose de résoudre ces problèmes en définissant un DSL indépendant de toute technologie qui capture l'essence de la répartition de charge, et en utilisant des transformations de modèles pour générer automatiquement les configurations ou le code cible.

Chapitre 3

Tableau des concepts identifiés

Le tableau suivant recense les concepts clés du domaine de la répartition de charge qui seront modélisés dans notre métamodèle :

N°	Concept	Description	Attributs
1	LoadBalancerSystem	Système global, élément racine	name, description, version
2	Cluster	Groupe logique de serveurs backend	name, maxConnections
3	Server	Serveur backend individuel	name, host, port, weight, maxConnections, enabled
4	LoadBalancer	Composant répartiteur	name, algorithm, stickySession
5	Algorithm	Énumération des algorithmes	ROUND_ROBIN, WEIGHTED_RR, LEAST_CONN, IP_HASH, RANDOM
6	HealthCheck	Vérification de disponibilité	protocol, path, interval, timeout, thresholds
7	Listener	Point d'écoute réseau	name, protocol, port, targetCluster
8	Protocol	Protocoles supportés	HTTP, HTTPS, TCP, UDP
9	SessionPersistence	Affinité de session	type, cookieName, ttl
10	Alert	Règle d'alerte	name, metric, threshold, action
11	ScalingRule	Mise à l'échelle auto	name, metric, scaleUp/Down, min/maxInstances
12	Metric	Métriques surveillées	CPU, MEMORY, REQUESTS, RESPONSE_TIME, CONNECTIONS

Chapitre 4

Règles de gestion et exigences

Les règles suivantes doivent être respectées dans tout modèle conforme à notre métamodèle :

Code	**Règle**	**Description**
RG01	Unicité des noms de serveurs	Tous les serveurs d'un même cluster doivent avoir des noms uniques
RG02	Port valide	Le port doit être compris entre 1 et 65535
RG03	Poids positif	Le poids d'un serveur doit être ≥ 1
RG04	Cluster non vide	Un cluster doit contenir au moins un serveur
RG05	Health check cohérent	L'intervalle doit être supérieur au timeout
RG06	Seuils de scaling cohérents	Le seuil de scale-up > seuil de scale-down
RG07	Instances min/max	minInstances $\leq$ maxInstances
RG08	Listener unique par port	Deux listeners ne peuvent pas écouter sur le même port
RG09	Weighted requiert des poids	Si algorithme = Weighted RR, tous les serveurs doivent avoir un poids ≥ 1
RG10	Cookie name requis	Si persistence = COOKIE, cookieName doit être renseigné
RG11	Au moins un listener	Le système doit avoir au moins un listener
RG12	Serveur actif requis	Un cluster doit avoir au moins un serveur enabled = true

Chapitre 5

Décomposition en modules

5.1 Architecture des packages

Notre système est organisé en modules pour une meilleure séparation des préoccupations :

```
loadbalancer.dsl/
  metamodel/                    # Metamodele
    core/                       # Concepts fondamentaux
      LoadBalancerSystem, Cluster, Server
    balancing/                  # Repartition
      LoadBalancer, Algorithm, SessionPersistence
    networking/                 # Reseau
      Listener, Protocol
    monitoring/                 # Surveillance
      HealthCheck, Alert, Metric
    scaling/                    # Mise a l'echelle
      ScalingRule
  constraints/                  # Contraintes OCL
  concrete.syntax/              # Syntaxe concrete (Xtext)
  transformations/
    m2m/                        # Modele-a-modele (ATL)
    m2t/                        # Modele-a-texte (Acceleo)
  examples/                     # Etudes de cas
```

Listing 5.1 – Structure en packages du projet

5.2 Description des modules

Table 5.1 – Description des modules du système

Module	**Rôle**	**Contenu**
metamodel.core	Concepts fondamentaux	LoadBalancerSystem, Cluster, Server
metamodel.balancing	Logique de répartition	LoadBalancer, Algorithm, SessionPersistence
metamodel.networking	Aspects réseau	Listener, Protocol
metamodel.monitoring	Surveillance et alertes	HealthCheck, Alert, Metric
metamodel.scaling	Mise à l'échelle	ScalingRule
constraints	Contraintes OCL	Fichiers .ocl
concrete.syntax	Grammaire Xtext	Fichier .xtext
transformations.m2m	Transfo M2M	Fichiers .atl
transformations.m2t	Transfo M2T	Fichiers .mtl

Chapitre 6

Schéma du métamodèle

6.1 Diagramme global du métamodèle

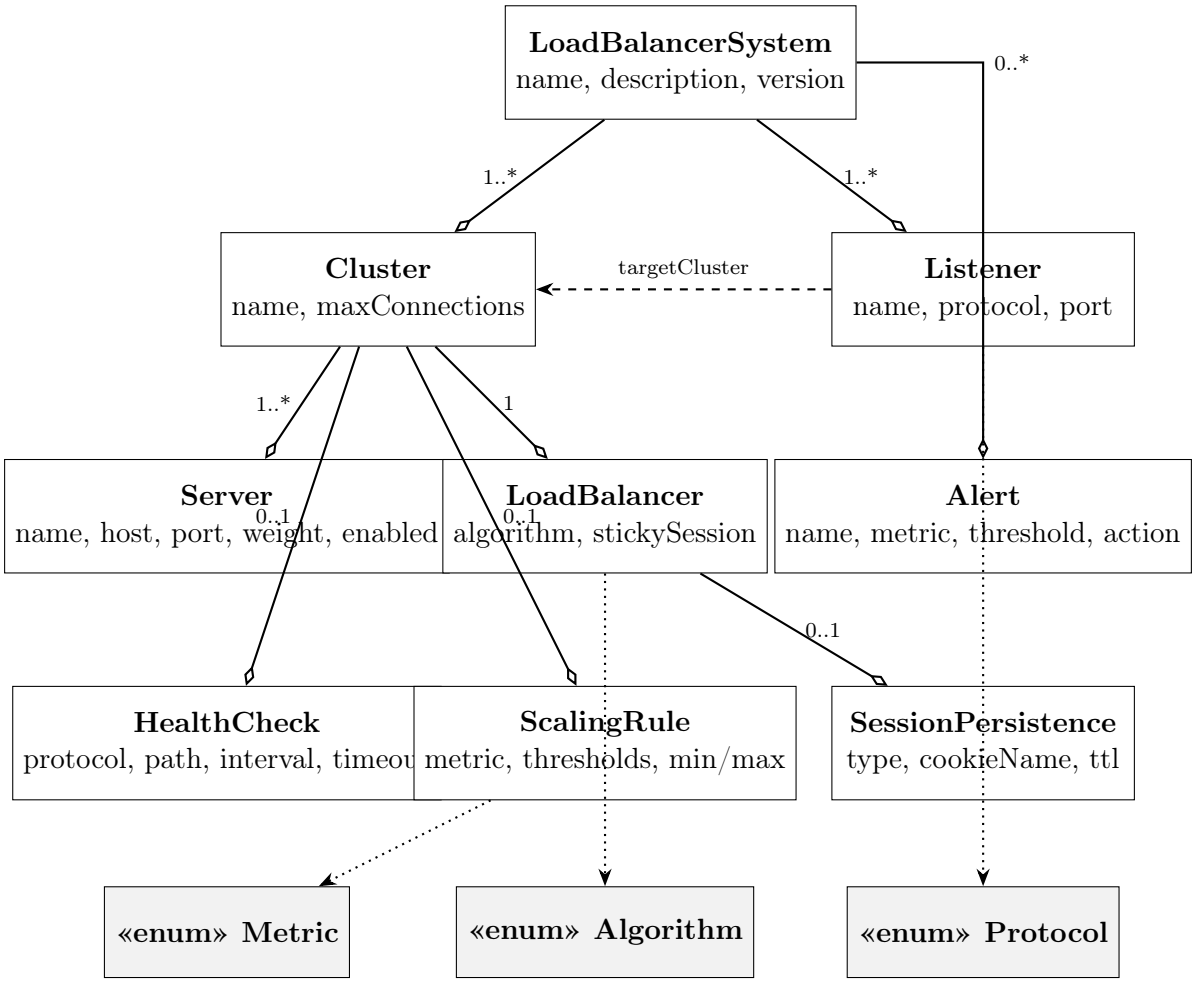

FIGURE 6.1 – Diagramme de classes du métamodèle global

6.2 Module Core – Détail

```
<?xml version="1.0" encoding="UTF-8"?>
```

```xml
<ecore:EPackage xmi:version="2.0"
    xmlns:xmi="http://www.omg.org/XMI"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:ecore="http://www.eclipse.org/emf/2002/Ecore"
    name="loadbalancer"
    nsURI="http://www.example.org/loadbalancer"
    nsPrefix="lb">

  <eClassifiers xsi:type="ecore:EClass" name="LoadBalancerSystem">
    <eStructuralFeatures xsi:type="ecore:EAttribute" name="name"
        eType="ecore:EDataType
           http://www.eclipse.org/emf/2002/Ecore#//EString"/>
    <eStructuralFeatures xsi:type="ecore:EAttribute"
       name="description"
        eType="ecore:EDataType
           http://www.eclipse.org/emf/2002/Ecore#//EString"/>
    <eStructuralFeatures xsi:type="ecore:EAttribute" name="version"
        eType="ecore:EDataType
           http://www.eclipse.org/emf/2002/Ecore#//EString"/>
    <eStructuralFeatures xsi:type="ecore:EReference"
       name="clusters"
        upperBound="-1" eType="#//Cluster" containment="true"/>
    <eStructuralFeatures xsi:type="ecore:EReference"
       name="listeners"
        upperBound="-1" eType="#//Listener" containment="true"/>
    <eStructuralFeatures xsi:type="ecore:EReference" name="alerts"
        upperBound="-1" eType="#//Alert" containment="true"/>
  </eClassifiers>

  <eClassifiers xsi:type="ecore:EClass" name="Cluster">
    <eStructuralFeatures xsi:type="ecore:EAttribute" name="name"
        eType="ecore:EDataType
           http://www.eclipse.org/emf/2002/Ecore#//EString"/>
    <eStructuralFeatures xsi:type="ecore:EReference" name="servers"
        upperBound="-1" eType="#//Server" containment="true"/>
    <eStructuralFeatures xsi:type="ecore:EReference"
       name="loadBalancer"
        eType="#//LoadBalancer" containment="true"/>
    <eStructuralFeatures xsi:type="ecore:EReference"
       name="healthCheck"
        eType="#//HealthCheck" containment="true"/>
    <eStructuralFeatures xsi:type="ecore:EReference"
       name="scalingRule"
        eType="#//ScalingRule" containment="true"/>
  </eClassifiers>

  <eClassifiers xsi:type="ecore:EClass" name="Server">
    <eStructuralFeatures xsi:type="ecore:EAttribute" name="name"
        eType="ecore:EDataType
           http://www.eclipse.org/emf/2002/Ecore#//EString"/>
    <eStructuralFeatures xsi:type="ecore:EAttribute" name="host"
```

```
        eType="ecore:EDataType http://www.eclipse.org/emf/2002/Ecore#//EString"/>
    <eStructuralFeatures xsi:type="ecore:EAttribute" name="port"
        eType="ecore:EDataType http://www.eclipse.org/emf/2002/Ecore#//EInt"/>
    <eStructuralFeatures xsi:type="ecore:EAttribute" name="weight"
        eType="ecore:EDataType http://www.eclipse.org/emf/2002/Ecore#//EInt"
        defaultValueLiteral="1"/>
    <eStructuralFeatures xsi:type="ecore:EAttribute" name="maxConnections"
        eType="ecore:EDataType http://www.eclipse.org/emf/2002/Ecore#//EInt"/>
    <eStructuralFeatures xsi:type="ecore:EAttribute" name="enabled"
        eType="ecore:EDataType http://www.eclipse.org/emf/2002/Ecore#//EBoolean"
        defaultValueLiteral="true"/>
  </eClassifiers>

  <eClassifiers xsi:type="ecore:EEnum" name="Algorithm">
    <eLiterals name="ROUND_ROBIN"/>
    <eLiterals name="WEIGHTED_ROUND_ROBIN" value="1"/>
    <eLiterals name="LEAST_CONNECTIONS" value="2"/>
    <eLiterals name="IP_HASH" value="3"/>
    <eLiterals name="RANDOM" value="4"/>
  </eClassifiers>

  <eClassifiers xsi:type="ecore:EEnum" name="Protocol">
    <eLiterals name="HTTP"/>
    <eLiterals name="HTTPS" value="1"/>
    <eLiterals name="TCP" value="2"/>
    <eLiterals name="UDP" value="3"/>
  </eClassifiers>

  <eClassifiers xsi:type="ecore:EEnum" name="Metric">
    <eLiterals name="CPU_USAGE"/>
    <eLiterals name="MEMORY_USAGE" value="1"/>
    <eLiterals name="REQUEST_COUNT" value="2"/>
    <eLiterals name="RESPONSE_TIME" value="3"/>
    <eLiterals name="ACTIVE_CONNECTIONS" value="4"/>
  </eClassifiers>

</ecore:EPackage>
```

Listing 6.1 – Extrait du métamodèle Ecore (loadbalancer.ecore)

Chapitre 7

Contraintes OCL

Chaque règle de gestion est formalisée en OCL (*Object Constraint Language*) :

7.1 RG01 – Unicité des noms de serveurs

```
context Cluster
inv uniqueServerNames:
  self.servers->isUnique(s | s.name)
```

7.2 RG02 – Port valide

```
context Server
inv validPort:
  self.port >= 1 and self.port <= 65535

context Listener
inv validListenerPort:
  self.port >= 1 and self.port <= 65535
```

7.3 RG03 – Poids positif

```
context Server
inv positiveWeight:
  self.weight >= 1
```

7.4 RG04 – Cluster non vide

```
context Cluster
inv nonEmptyCluster:
  self.servers->notEmpty()
```

7.5 RG05 – Health check cohérent

```
context HealthCheck
inv intervalGreaterThanTimeout:
  self.interval > self.timeout
```

7.6 RG06 – Seuils de scaling cohérents

```
context ScalingRule
inv scaleUpGreaterThanDown:
  self.scaleUpThreshold > self.scaleDownThreshold
```

7.7 RG07 – Instances min/max cohérentes

```
context ScalingRule
inv minLessOrEqualMax:
  self.minInstances <= self.maxInstances
```

7.8 RG08 – Listener unique par port

```
context LoadBalancerSystem
inv uniqueListenerPorts:
  self.listeners->isUnique(l | l.port)
```

7.9 RG09 – Algorithme Weighted requiert des poids

```
context Cluster
inv weightedRequiresWeights:
  self.loadBalancer.algorithm = Algorithm::WEIGHTED_ROUND_ROBIN
  implies self.servers->forAll(s | s.weight >= 1)
```

7.10 RG10 – Session Persistence cookie

```
context SessionPersistence
inv cookieRequiresName:
  self.type = PersistenceType::COOKIE
  implies self.cookieName <> '' and self.cookieName <> null
```

7.11 RG11 – Au moins un listener

```
context LoadBalancerSystem
inv atLeastOneListener:
  self.listeners->size() >= 1
```

7.12 RG12 – Serveur actif dans le cluster

```
context Cluster
inv atLeastOneEnabledServer:
  self.servers->exists(s | s.enabled = true)
```

Chapitre 8

Niveaux MDA et architecture de l'approche

8.1 Les niveaux MDA pris en compte

Table 8.1 – Niveaux MDA et artefacts produits

Niveau	**Description**	**Artefact produit**
CIM	Modèle indépendant de l'informatique	Description du domaine, concepts, règles
PIM	Modèle indépendant de la plateforme	Métamodèle Ecore, contraintes OCL, modèles DSL (.lb)
PSM	Modèle spécifique plateforme	Modèle intermédiaire Nginx/HAProxy (transfo ATL)
Code	Code exécutable généré	nginx.conf, docker-compose.yml (transfo Acceleo)

8.2 Schéma d'architecture de l'approche

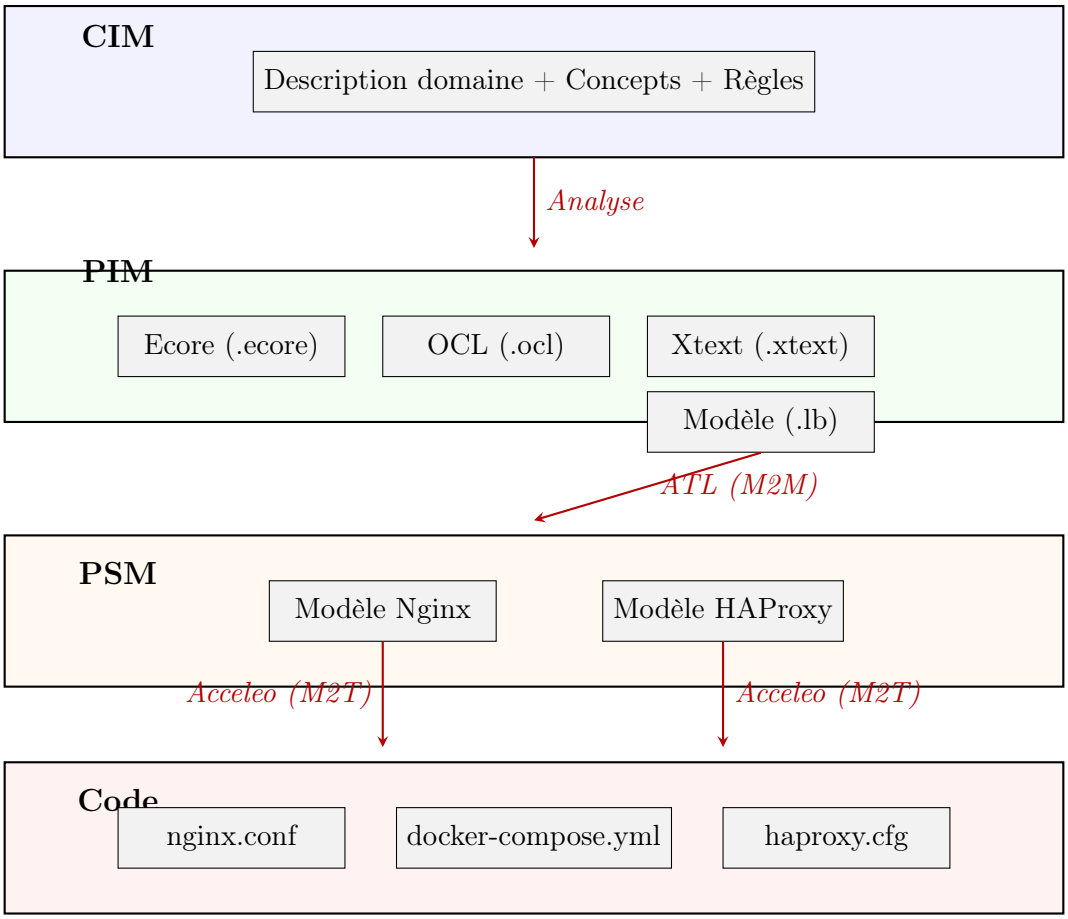

FIGURE 8.1 – Architecture MDA de l'approche

8.3 Justification des choix technologiques

TABLE 8.2 – Outils et technologies utilisés

Outil	**Rôle**	**Justification**
EMF / Ecore	Métamodèle (syntaxe abstraite)	Standard pour la méta-modélisation Eclipse
OCL	Contraintes	Langage standardisé OMG pour contraintes
Xtext	Syntaxe concrète DSL	Framework mature, éditeur intégré Eclipse
ATL	Transformations M2M	Langage hybride déclaratif/impératif
Acceleo	Transformations M2T	Génération basée templates, compatible EMF

Chapitre 9

Spécification des transformations

9.1 Vue d'ensemble

Table 9.1 – Transformations définies

Transfo	Type	Source	Cible	Outil
T1 : PIM→PSM Nginx	M2M	Modèle .lb	Modèle Nginx	ATL
T2 : PIM→PSM HAProxy	M2M	Modèle .lb	Modèle HAProxy	ATL
T3 : PSM Nginx→Code	M2T	Modèle Nginx	nginx.conf	Acceleo
T4 : PSM HAProxy→Code	M2T	Modèle HAProxy	haproxy.cfg	Acceleo
T5 : PIM→Docker	M2T	Modèle .lb	docker-compose.yml	Acceleo

9.2 Définition de la grammaire Xtext (syntaxe concrète)

La grammaire Xtext définit la syntaxe concrète textuelle de notre DSL. Elle permet de créer un éditeur avec coloration syntaxique, auto-complétion et validation intégrée dans Eclipse. Le fichier de grammaire est `LoadBalancer.xtext` :

```
grammar org.loadbalancer.dsl.LoadBalancer
  with org.eclipse.xtext.common.Terminals

generate loadBalancer
   "http://www.loadbalancer.org/dsl/LoadBalancer"

// ==========================================
// Racine du modele
// ==========================================
LoadBalancerSystem:
  'LoadBalancerSystem' name=STRING '{'
```

```
    ('description:' description=STRING)?
    ('version:' version=STRING)?
    clusters+=Cluster+
    listeners+=Listener+
    (alerts+=Alert)*
  '}';

// ==========================================
// Cluster de serveurs
// ==========================================
Cluster:
  'cluster' name=ID '{'
    loadBalancer=LoadBalancerConfig
    servers+=Server+
    (healthCheck=HealthCheck)?
    (scalingRule=ScalingRule)?
  '}';

// ==========================================
// Configuration du repartiteur
// ==========================================
LoadBalancerConfig:
  'loadbalancer' '{'
    'algorithm:' algorithm=Algorithm
    'stickySession:' stickySession=BooleanValue
    (sessionPersistence=SessionPersistence)?
  '}';

// ==========================================
// Serveur
// ==========================================
Server:
  'server' name=ID '{'
    'host:' host=STRING
    'port:' port=INT
    ('weight:' weight=INT)?
    ('maxConnections:' maxConnections=INT)?
    'enabled:' enabled=BooleanValue
  '}';

// ==========================================
// Health Check
// ==========================================
HealthCheck:
  'healthCheck' '{'
    'protocol:' protocol=Protocol
    'path:' path=STRING
    'interval:' interval=INT
    'timeout:' timeout=INT
    ('healthyThreshold:' healthyThreshold=INT)?
    ('unhealthyThreshold:' unhealthyThreshold=INT)?
```

```
  '}';

// ==========================================
// Persistence de session
// ==========================================
SessionPersistence:
  'sessionPersistence' '{'
    'type:' type=PersistenceType
    ('cookieName:' cookieName=STRING)?
    ('ttl:' ttl=INT)?
  '}';

// ==========================================
// Regle de mise a l'echelle
// ==========================================
ScalingRule:
  'scalingRule' name=ID '{'
    'metric:' metric=Metric
    'scaleUpThreshold:' scaleUpThreshold=DOUBLE
    'scaleDownThreshold:' scaleDownThreshold=DOUBLE
    'minInstances:' minInstances=INT
    'maxInstances:' maxInstances=INT
  '}';

// ==========================================
// Listener (point d'entree)
// ==========================================
Listener:
  'listener' name=ID '{'
    'protocol:' protocol=Protocol
    'port:' port=INT
    'targetCluster:' targetCluster=[Cluster]
  '}';

// ==========================================
// Alerte
// ==========================================
Alert:
  'alert' name=ID '{'
    'metric:' metric=Metric
    'threshold:' threshold=DOUBLE
    'action:' action=STRING
  '}';

// ==========================================
// Enumerations
// ==========================================
enum Algorithm:
  ROUND_ROBIN = 'ROUND_ROBIN' |
  WEIGHTED_ROUND_ROBIN = 'WEIGHTED_ROUND_ROBIN' |
  LEAST_CONNECTIONS = 'LEAST_CONNECTIONS' |
```

```
  IP_HASH = 'IP_HASH' |
  RANDOM = 'RANDOM';

enum Protocol:
  HTTP = 'HTTP' |
  HTTPS = 'HTTPS' |
  TCP = 'TCP';

enum Metric:
  CPU_USAGE = 'CPU_USAGE' |
  MEMORY_USAGE = 'MEMORY_USAGE' |
  RESPONSE_TIME = 'RESPONSE_TIME' |
  REQUEST_COUNT = 'REQUEST_COUNT' |
  ERROR_RATE = 'ERROR_RATE';

enum PersistenceType:
  COOKIE = 'COOKIE' |
  IP = 'IP' |
  HEADER = 'HEADER';

// ==========================================
// Types terminaux
// ==========================================
BooleanValue:
  value=('true' | 'false');

terminal DOUBLE returns ecore::EDouble:
  INT '.' INT;
```

Listing 9.1 – Grammaire Xtext complète du DSL de répartition de charge

9.2.1 Explication de la grammaire

La grammaire ci-dessus définit la syntaxe concrète complète du DSL. Voici les éléments clés :

- **LoadBalancerSystem** : élément racine, contient les clusters, listeners et alertes.
- **Cluster** : regroupe un ensemble de serveurs avec une stratégie de répartition commune. Chaque cluster possède une configuration de `LoadBalancerConfig`, au moins un `Server`, et optionnellement un `HealthCheck` et un `ScalingRule`.
- **Server** : représente un serveur backend avec son hôte, port, poids et état d'activation.
- **Listener** : définit un point d'écoute (port d'entrée) et le cluster cible via une **référence croisée** (`[Cluster]`).
- **Énumérations** : `Algorithm`, `Protocol`, `Metric` et `PersistenceType` contraignent les valeurs acceptées.
- **Terminal DOUBLE** : terminal personnalisé pour les valeurs décimales (seuils de scaling et alertes).

9.2.2 Artefacts générés par Xtext

L'exécution de `Generate Xtext Artifacts` produit automatiquement :

TABLE 9.2 – Artefacts générés par le workflow Xtext

Artefact	Description
Parseur ANTLR	Analyse syntaxique des fichiers `.lb`
Modèle EMF (EPackage)	Syntaxe abstraite dérivée de la grammaire
Éditeur Eclipse	Coloration syntaxique, auto-complétion, outline
Sérialiseur	Conversion modèle ↔ texte
Validateur	Classe `LoadBalancerValidator.xtend` pour les contraintes
Formateur	Indentation et formatage du code DSL
Scope Provider	Résolution des références croisées (ex : `targetCluster`)

9.3 Transformation T1 : PIM → PSM Nginx (ATL)

```
-- Fichier : PIM2Nginx.atl
module PIM2Nginx;
create OUT : NginxModel from IN : LoadBalancerModel;

rule System2NginxConfig {
  from s : LoadBalancerModel!LoadBalancerSystem
  to   c : NginxModel!NginxConfig (
    workerProcesses <- 'auto',
    workerConnections <- 1024,
    upstreams <- s.clusters->collect(cl |
      thisModule.Cluster2Upstream(cl)),
    servers <- s.listeners->collect(l |
      thisModule.Listener2ServerBlock(l))
  )
}

lazy rule Cluster2Upstream {
  from cl : LoadBalancerModel!Cluster
  to   u : NginxModel!Upstream (
    name <- cl.name,
    algorithm <- cl.loadBalancer.algorithm.mapNginxAlgo(),
    servers <- cl.servers->select(s | s.enabled)
              ->collect(s | thisModule.Server2UpstreamServer(s))
  )
}

lazy rule Server2UpstreamServer {
  from s : LoadBalancerModel!Server
  to   us : NginxModel!UpstreamServer (
    address <- s.host + ':' + s.port.toString(),
```

```
    weight <- s.weight
  )
}

lazy rule Listener2ServerBlock {
  from l : LoadBalancerModel!Listener
  to   sb : NginxModel!ServerBlock (
    listenPort <- l.port,
    proxyPass <- 'http://' + l.targetCluster.name
  )
}

helper context LoadBalancerModel!Algorithm
def : mapNginxAlgo() : String =
  if self = #LEAST_CONNECTIONS then 'least_conn'
  else if self = #IP_HASH then 'ip_hash'
  else '' endif endif;
```

Listing 9.2 – Transformation ATL PIM vers Nginx

9.4 Transformation T3 : PSM Nginx → Code (Acceleo)

```
[comment encoding = UTF-8 /]
[module generateNginx('http://www.example.org/nginx')]

[template public generateNginxConf(config : NginxConfig)]
[comment @main /]
[file ('nginx.conf', false, 'UTF-8')]
worker_processes [config.workerProcesses/];

events {
    worker_connections [config.workerConnections/];
}

http {
[for (upstream : Upstream | config.upstreams)]
    upstream [upstream.name/] {
[if (upstream.algorithm = 'least_conn')]
        least_conn;
[elseif (upstream.algorithm = 'ip_hash')]
        ip_hash;
[/if]
[for (server : UpstreamServer | upstream.servers)]
        server [server.address/] weight=[server.weight/];
[/for]
    }

[/for]
[for (server : ServerBlock | config.servers)]
    server {
```

```
        listen [server.listenPort/];
        location / {
            proxy_pass [server.proxyPass/];
        }
    }
[/for]
}
[/file]
[/template]
```

Listing 9.3 – Template Acceleo pour nginx.conf

9.5 Transformation T5 : PIM → Docker Compose (Acceleo)

```
[comment encoding = UTF-8 /]
[module
   generateDockerCompose('http://www.example.org/loadbalancer')]

[template public generateCompose(system : LoadBalancerSystem)]
[comment @main /]
[file ('docker-compose.yml', false, 'UTF-8')]
version: '3.8'
services:
[for (cluster : Cluster | system.clusters)]
[for (server : Server | cluster.servers)]
  [server.name/]:
    image: nginx:alpine
    ports:
      - "[server.port/]:[server.port/]"
[/for]
  [cluster.name/]-lb:
    image: nginx:alpine
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
    ports:
[for (listener : Listener | system.listeners)]
[if (listener.targetCluster.name = cluster.name)]
      - "[listener.port/]:[listener.port/]"
[/if]
[/for]
    depends_on:
[for (server : Server | cluster.servers)]
      - [server.name/]
[/for]
[/for]
[/file]
[/template]
```

Listing 9.4 – Template Acceleo pour docker-compose.yml

Chapitre 10

Installation et configuration des outils

Cette section décrit l'installation des outils nécessaires pour exécuter et tester les artefacts générés par notre chaîne de transformation (nginx.conf et docker-compose.yml).

10.1 Prérequis système

- Système d'exploitation : Linux (Ubuntu/Debian recommandé), macOS ou Windows avec WSL2
- Minimum 4 Go de RAM disponible
- Connexion internet pour télécharger les images Docker

10.2 Installation de Docker

Docker est le moteur de conteneurisation qui permet d'exécuter les serveurs backend et le load balancer.

10.2.1 Sur Ubuntu/Debian

```
# 1. Mise a jour du systeme
sudo apt update && sudo apt upgrade -y

# 2. Installation des dependances
sudo apt install -y apt-transport-https ca-certificates \
  curl gnupg lsb-release

# 3. Ajout de la cle GPG officielle Docker
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o
     /usr/share/keyrings/docker-archive-keyring.gpg

# 4. Ajout du depot Docker
echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] \
  https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable" | \
```

```
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# 5. Installation de Docker Engine
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io

# 6. Ajout de l'utilisateur au groupe docker (evite sudo)
sudo usermod -aG docker $USER
newgrp docker

# 7. Verification
docker --version
docker run hello-world
```

Listing 10.1 – Installation de Docker sur Ubuntu/Debian

10.2.2 Sur macOS

```
# Telecharger Docker Desktop depuis :
# https://www.docker.com/products/docker-desktop/
# Ou via Homebrew :
brew install --cask docker

# Lancer Docker Desktop puis verifier :
docker --version
```

Listing 10.2 – Installation de Docker sur macOS

10.3 Installation de Docker Compose

Docker Compose V2 est inclus avec Docker Desktop. Sur Linux, il peut être installé séparément :

```
# Docker Compose V2 (plugin Docker)
sudo apt install -y docker-compose-plugin

# Verification
docker compose version

# Alternative : installation standalone
sudo curl -SL
   https://github.com/docker/compose/releases/latest/download/docker-compose-l
   \
  -o /usr/local/bin/docker-compose
sudo chmod +x /usr/local/bin/docker-compose
docker-compose --version
```

Listing 10.3 – Installation de Docker Compose sur Linux

10.4 Installation de Nginx (pour tests locaux sans Docker)

Bien que notre approche utilise Docker, il est utile d'avoir Nginx installé localement pour des tests rapides :

```
# Ubuntu/Debian
sudo apt install -y nginx

# Verification
nginx -v
sudo systemctl status nginx

# Test de configuration
sudo nginx -t

# Demarrage
sudo systemctl start nginx
sudo systemctl enable nginx
```

Listing 10.4 – Installation de Nginx

10.5 Installation des outils IDM (Eclipse)

Pour la partie modélisation et transformation :

```
# 1. Telecharger Eclipse Modeling Tools depuis :
#    https://www.eclipse.org/downloads/packages/
#    Choisir "Eclipse Modeling Tools"

# 2. Installer les plugins via Help > Eclipse Marketplace :
#    - Xtext Complete SDK
#    - ATL/EMFTVM
#    - Acceleo
#    - OCL Tools (Eclipse OCL)

# 3. Ou via Help > Install New Software :
#    URL: https://download.eclipse.org/releases/2023-12
#    Selectionner : Modeling > EMF, Xtext, ATL, Acceleo, OCL
```

Listing 10.5 – Installation d'Eclipse Modeling Tools

10.6 Outils complémentaires utiles

```
# curl - pour tester les requetes HTTP
sudo apt install -y curl

# jq - pour formatter les reponses JSON
sudo apt install -y jq

# htop - pour surveiller les ressources
```

```
sudo apt install -y htop

# ab (Apache Bench) - pour les tests de charge
sudo apt install -y apache2-utils

# wrk - outil de benchmark HTTP performant
sudo apt install -y wrk
```

Listing 10.6 – Outils complémentaires

10.7 Vérification de l'installation complète

```
#!/bin/bash
echo "=== Verification de l'installation ==="
echo ""
echo -n "Docker: "
docker --version 2>/dev/null || echo "NON INSTALLE"
echo -n "Docker Compose: "
docker compose version 2>/dev/null || echo "NON INSTALLE"
echo -n "Nginx: "
nginx -v 2>&1 || echo "NON INSTALLE"
echo -n "curl: "
curl --version | head -1 || echo "NON INSTALLE"
echo ""
echo "=== Test Docker ==="
docker run --rm hello-world 2>/dev/null && echo "Docker OK" ||
    echo "Docker ERREUR"
```

Listing 10.7 – Script de vérification

Chapitre 11

Étude de cas : Application E-Commerce

11.1 Description du cas d'étude

Nous illustrons notre approche avec un cas concret : une application web e-commerce déployée sur un système distribué. L'application possède :

- Un cluster de 3 serveurs web (frontend) avec répartition Round Robin
- Un cluster de 2 serveurs API (backend) avec répartition Least Connections
- Des health checks HTTP sur chaque cluster
- Une règle de scaling automatique basée sur l'utilisation CPU
- Des alertes en cas de dépassement de seuils

11.2 Modèle DSL du cas d'étude

Voici le modèle écrit avec notre DSL (syntaxe concrète textuelle) :

```
LoadBalancerSystem "ECommerceSystem" {
  description: "Systeme de repartition de charge pour e-commerce"
  version: "1.0"

  cluster WebFrontend {
    loadbalancer {
      algorithm: ROUND_ROBIN
      stickySession: true
      sessionPersistence {
        type: COOKIE
        cookieName: "SESSIONID"
        ttl: 3600
      }
    }

    server web1 {
      host: "192.168.1.10"
      port: 8080
      weight: 3
      maxConnections: 1000
      enabled: true
```

```
    }
    server web2 {
      host: "192.168.1.11"
      port: 8080
      weight: 2
      maxConnections: 800
      enabled: true
    }
    server web3 {
      host: "192.168.1.12"
      port: 8080
      weight: 1
      maxConnections: 500
      enabled: true
    }

    healthCheck {
      protocol: HTTP
      path: "/health"
      interval: 30
      timeout: 10
      healthyThreshold: 3
      unhealthyThreshold: 2
    }

    scalingRule autoScaleWeb {
      metric: CPU_USAGE
      scaleUpThreshold: 80.0
      scaleDownThreshold: 30.0
      minInstances: 2
      maxInstances: 10
    }
  }

  cluster APIBackend {
    loadbalancer {
      algorithm: LEAST_CONNECTIONS
      stickySession: false
    }

    server api1 {
      host: "192.168.2.10"
      port: 3000
      weight: 1
      maxConnections: 500
      enabled: true
    }
    server api2 {
      host: "192.168.2.11"
      port: 3000
      weight: 1
```

```
      maxConnections: 500
      enabled: true
    }

    healthCheck {
      protocol: HTTP
      path: "/api/health"
      interval: 15
      timeout: 5
      healthyThreshold: 2
      unhealthyThreshold: 3
    }
  }

  listener httpListener {
    protocol: HTTP
    port: 80
    targetCluster: WebFrontend
  }

  listener apiListener {
    protocol: HTTP
    port: 3000
    targetCluster: APIBackend
  }

  alert highCPU {
    metric: CPU_USAGE
    threshold: 90.0
    action: "email:admin@ecommerce.com"
  }

  alert highLatency {
    metric: RESPONSE_TIME
    threshold: 5000.0
    action: "webhook:https://hooks.slack.com/alert"
  }
}
```

Listing 11.1 – Modèle DSL de l'application e-commerce (ecommerce.lb)

11.3 Code généré : nginx.conf

Après exécution de la chaîne de transformations, le fichier nginx.conf suivant est généré :

```
worker_processes auto;

events {
    worker_connections 1024;
}
```

```

http {
    upstream WebFrontend {
        server 192.168.1.10:8080 weight=3;
        server 192.168.1.11:8080 weight=2;
        server 192.168.1.12:8080 weight=1;
    }

    upstream APIBackend {
        least_conn;
        server 192.168.2.10:3000 weight=1;
        server 192.168.2.11:3000 weight=1;
    }

    server {
        listen 80;
        location / {
            proxy_pass http://WebFrontend;
        }
    }

    server {
        listen 3000;
        location / {
            proxy_pass http://APIBackend;
        }
    }
}
```

Listing 11.2 – Fichier nginx.conf généré

11.4 Code généré : docker-compose.yml

```
version: '3.8'

services:
  web1:
    build: ./servers/web
    environment:
      - SERVER_NAME=web1
      - SERVER_PORT=8080
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
      interval: 30s
      timeout: 10s
      retries: 3

  web2:
    build: ./servers/web
    environment:
```

```
      - SERVER_NAME=web2
      - SERVER_PORT=8080
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
      interval: 30s
      timeout: 10s
      retries: 3

  web3:
    build: ./servers/web
    environment:
      - SERVER_NAME=web3
      - SERVER_PORT=8080
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
      interval: 30s
      timeout: 10s
      retries: 3

  api1:
    build: ./servers/api
    environment:
      - SERVER_NAME=api1
      - SERVER_PORT=3000
    healthcheck:
      test: ["CMD", "curl", "-f",
         "http://localhost:3000/api/health"]
      interval: 15s
      timeout: 5s
      retries: 3

  api2:
    build: ./servers/api
    environment:
      - SERVER_NAME=api2
      - SERVER_PORT=3000
    healthcheck:
      test: ["CMD", "curl", "-f",
         "http://localhost:3000/api/health"]
      interval: 15s
      timeout: 5s
      retries: 3

  loadbalancer:
    image: nginx:alpine
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
    ports:
      - "80:80"
      - "3000:3000"
    depends_on:
```

```
      - web1
      - web2
      - web3
      - api1
      - api2
```

Listing 11.3 – Fichier docker-compose.yml généré

11.5 Déploiement et test

11.5.1 Lancement de l'infrastructure

```
# Se placer dans le dossier du cas d'etude
cd case-study/

# Construire et demarrer tous les services
docker compose up --build -d

# Verifier que tous les conteneurs sont actifs
docker compose ps

# Voir les logs
docker compose logs -f
```

Listing 11.4 – Commandes de déploiement

11.5.2 Tests de la répartition de charge

```
# Test du cluster Web Frontend (Round Robin)
for i in $(seq 1 10); do
  echo "Request $i:"
  curl -s http://localhost:80/ | jq .
  echo "---"
done

# Test du cluster API Backend (Least Connections)
for i in $(seq 1 10); do
  echo "Request $i:"
  curl -s http://localhost:3000/api/health | jq .
  echo "---"
done

# Test de charge avec wrk
wrk -t4 -c100 -d30s http://localhost:80/

# Test de charge avec ab
ab -n 1000 -c 50 http://localhost:80/
```

Listing 11.5 – Tests de vérification

11.5.3 Vérification du health check

```
# Arreter un serveur pour tester le failover
docker compose stop web3

# Verifier que les requetes sont redistribuees
for i in $(seq 1 6); do
  curl -s http://localhost:80/ | jq .serverName
done

# Redemarrer le serveur
docker compose start web3
```

Listing 11.6 – Test du health check

Chapitre 12

Tutoriel de réalisation pas à pas

12.1 Prérequis

— Eclipse Modeling Tools (version 2023-12 ou supérieure)
— Plugins : EMF, Xtext, ATL, Acceleo, OCL Tools
— Java JDK 17+
— Docker et Docker Compose (pour les tests)

12.2 Étape 1 : Création du métamodèle Ecore

1. Dans Eclipse : `File` → `New` → `Other` → `Eclipse Modeling Framework` → `Ecore Modeling Project`
2. Nommer le projet : `org.loadbalancer.metamodel`
3. Créer le fichier `loadbalancer.ecore`
4. Définir toutes les classes, attributs et relations (voir chapitre 5)
5. Ouvrir avec l'éditeur graphique pour visualiser le diagramme

12.3 Étape 2 : Génération du code Java EMF

1. Clic droit sur `loadbalancer.ecore` → `New` → `EMF Generator Model`
2. Créer `loadbalancer.genmodel`
3. Ouvrir le `.genmodel` → Clic droit sur la racine → `Generate All`

12.4 Étape 3 : Définition de la syntaxe concrète (Xtext)

1. `File` → `New` → `Project` → `Xtext` → `Xtext Project`
2. Nommer : `org.loadbalancer.dsl`, extension : `.lb`
3. Définir la grammaire (voir chapitre 8, section Xtext)
4. Exécuter : Clic droit → `Run As` → `Generate Xtext Artifacts`
5. Tester dans une Eclipse Application

12.5 Étape 4 : Implémentation des contraintes

1. Ouvrir le fichier `LoadBalancerValidator.xtend` généré par Xtext
2. Ajouter les méthodes de validation (voir chapitre 6)
3. Chaque méthode annotée `@Check` implémente une règle OCL

12.6 Étape 5 : Transformations ATL

1. `File` → `New` → `ATL Project` : `org.loadbalancer.transformations`
2. Créer le métamodèle cible `nginx.ecore`
3. Écrire `PIM2Nginx.atl` (voir chapitre 8)
4. Configurer le `Run Configuration ATL`

12.7 Étape 6 : Génération de code (Acceleo)

1. `File` → `New` → `Acceleo Project` : `org.loadbalancer.generators`
2. Créer les templates `.mtl` (voir chapitre 8)
3. Configurer le launch configuration

12.8 Étape 7 : Test de bout en bout

1. Lancer une Eclipse Application
2. Créer un fichier `ecommerce.lb`
3. Vérifier la validation des contraintes
4. Exécuter les transformations
5. Récupérer les fichiers générés
6. Tester avec Docker Compose (chapitre 10)

Chapitre 13

Perspectives et améliorations

Ce qui resterait à faire pour atteindre pleinement les objectifs :

- Support complet HAProxy comme cible PSM supplémentaire
- Génération de manifestes Kubernetes (Deployment, Service, Ingress)
- Éditeur graphique avec Eclipse Sirius
- Support du load balancing DNS
- Intégration monitoring (Prometheus + Grafana)
- Validation dynamique avec simulation de charge
- Support multi-cloud (AWS ALB, GCP LB, Azure LB)
- Interface web pour les non-développeurs

Conclusion

Ce projet a permis de mettre en pratique les concepts fondamentaux de l'Ingénierie Dirigée par les Modèles (IDM) à travers la conception d'un DSL dédié à la répartition de charge dans les systèmes distribués.

Nous avons suivi une démarche MDA complète :

- **Niveau CIM** : analyse et description du domaine de la répartition de charge
- **Niveau PIM** : conception du métamodèle Ecore, définition des contraintes OCL, et création de la syntaxe concrète Xtext
- **Niveau PSM** : transformation ATL vers des modèles spécifiques (Nginx)
- **Niveau Code** : génération automatique de fichiers de configuration via Acceleo

L'approche proposée démontre les avantages de l'IDM : abstraction, réutilisabilité, portabilité entre plateformes, et validation formelle des modèles. Le DSL créé permet aux administrateurs système de décrire leur infrastructure de répartition de charge de manière déclarative et indépendante de la technologie, tout en bénéficiant d'une génération automatique de code fiable et cohérente.

L'étude de cas avec Docker Compose et Nginx prouve la faisabilité de l'approche en permettant un déploiement réel de l'infrastructure générée.

Références bibliographiques

[1] J. Bézivin, « On the Unification Power of Models », *Software and Systems Modeling*, vol. 4, no. 2, pp. 171–188, 2005.

[2] OMG, « MDA Guide Version 2.0 », Object Management Group, 2014.

[3] F. Jouault, F. Allilaire, J. Bézivin, I. Kurtev, « ATL : A model transformation tool », *Science of Computer Programming*, vol. 72, no. 1-2, pp. 31–39, 2008.

[4] M. Bettini, « Implementing Domain-Specific Languages with Xtext and Xtend », Packt Publishing, 2nd edition, 2016.

[5] Eclipse Foundation, « Acceleo User Guide », https://wiki.eclipse.org/Acceleo.

[6] Eclipse Foundation, « Eclipse Modeling Framework (EMF) », https://www.eclipse.org/modeling/emf/.

[7] Nginx Documentation, « HTTP Load Balancing », https://docs.nginx.com/.

[8] HAProxy Documentation, « Configuration Manual », https://www.haproxy.org/.

[9] T. Stahl, M. Voelter, « Model-Driven Software Development », Wiley, 2006.

[10] A. Kleppe, J. Warmer, W. Bast, « MDA Explained », Addison-Wesley, 2003.